COMP 4033 - Health Informatics Data Analytics: Assignment 1 – Data Analysis using R Programming

Submission Date: March 28, 2026

Group 1 Members:

Mary Joyce Zamora, Yeafi Awal, Janine Alyssa Atienza, Jhoanna Mae Agustin, Mencha Tembong, Ola Qutmeh, Sareea Waheed, Halil Ibrahim Kerim, Zubariya Siddiqui

Submitted to:

Professor Esther Shalini Rajasekaran and Professor Dhara Desai

Introduction

For this assignment, we used the heart disease dataset to explore how different patient characteristics may be related to heart disease. The dataset includes several clinical and demographic variables, such as age, sex, resting blood pressure, cholesterol, maximum heart rate, exercise-induced angina, and oldpeak. These variables make it possible to look at patterns and compare patients with and without heart disease. Using R Programming, the dataset was cleaned, organized, and analyzed using summary statistics, graphs, and Pearson correlation. The purpose of the analysis was to identify which variables seem to provide the most meaningful insight into heart disease in this dataset.

Datasource: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

Key Terms Used in the Dataset

Before presenting the analysis, a few key terms from the dataset are defined below for clarity. Some of the variable names are abbreviated clinical terms, so these definitions help make the results easier to understand.

- **target** – indicates whether heart disease is present (`1 = disease`, `0 = no disease`)
- **age** – age of the patient in years
- **sex** – sex of the patient (`1 = male`, `0 = female`)
- **cp** – chest pain type
- **trestbps** – resting blood pressure
- **chol** – serum cholesterol level
- **fbs** – fasting blood sugar greater than 120 mg/dL (`1 = true`, `0 = false`)
- **restecg** – resting electrocardiographic results
- **thalach** – maximum heart rate achieved during exercise
- **exang** – exercise-induced angina (`1 = yes`, `0 = no`)
- **oldpeak** – ST depression induced by exercise relative to rest
- **slope** – the slope of the peak exercise ST segment
- **ca** – number of major blood vessels colored by fluoroscopy

- **thal** – thalassemia-related test result used in heart disease assessment

Important to understand:

- **ST segment** - is a part of the ECG wave that helps show how the heart is responding electrically after a heartbeat. Abnormal changes in this part of the tracing can sometimes indicate heart-related problems.
- **ST depression** - is when the ST segment on an ECG drops below its usual level. This may indicate an abnormal heart response during exercise or stress.

1. Loading the Libraries and Dataset

```
library(dplyr)
library(tidyr)
library(ggplot2)

heart <- read.csv("heart.csv")
```

2. Print the Structure of the Dataset

```
str(heart)
```

```
## 'data.frame':    1025 obs. of  14 variables:
##  $ age     : int  52 53 70 61 62 58 58 55 46 54 ...
##  $ sex     : int  1 1 1 1 0 0 1 1 1 1 ...
##  $ cp      : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ trestbps: int  125 140 145 148 138 100 114 160 120 122 ...
##  $ chol    : int  212 203 174 203 294 248 318 289 249 286 ...
##  $ fbs     : int  0 1 0 0 1 0 0 0 0 0 ...
##  $ restecg : int  1 0 1 1 1 0 2 0 0 0 ...
##  $ thalach : int  168 155 125 161 106 122 140 145 144 116 ...
##  $ exang   : int  0 1 1 0 0 0 0 1 0 1 ...
##  $ oldpeak : num  1 3.1 2.6 0 1.9 1 4.4 0.8 0.8 3.2 ...
##  $ slope   : int  2 0 0 2 1 1 0 1 2 1 ...
##  $ ca      : int  2 0 0 1 3 0 3 1 0 2 ...
##  $ thal    : int  3 3 3 3 2 2 1 3 3 2 ...
##  $ target  : int  0 0 0 0 0 1 0 0 0 0 ...
```

The str() function shows the dataset's structure, including 1025 observations and 14 variables. Most are numeric integers

3. List the Variables in the Dataset

```
names(heart)
```

```
##  [1] "age"      "sex"      "cp"       "trestbps" "chol"     "fbs"
##  [7] "restecg"  "thalach"  "exang"    "oldpeak"  "slope"    "ca"
## [13] "thal"     "target"
```

The names() function lists all 14 column names in the dataset, representing the clinical attributes recorded during patients' cardiac assessments.

4. Print the Top 15 Rows of the Dataset

```
head(heart, 15)
```

```
##    age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal
## 1   52   1  0      125  212   0       1     168     0     1.0     2  2    3
## 2   53   1  0      140  203   1       0     155     1     3.1     0  0    3
## 3   70   1  0      145  174   0       1     125     1     2.6     0  0    3
## 4   61   1  0      148  203   0       1     161     0     0.0     2  1    3
## 5   62   0  0      138  294   1       1     106     0     1.9     1  3    2
## 6   58   0  0      100  248   0       0     122     0     1.0     1  0    2
## 7   58   1  0      114  318   0       2     140     0     4.4     0  3    1
## 8   55   1  0      160  289   0       0     145     1     0.8     1  1    3
## 9   46   1  0      120  249   0       0     144     0     0.8     2  0    3
## 10  54   1  0      122  286   0       0     116     1     3.2     1  2    2
## 11  71   0  0      112  149   0       1     125     0     1.6     1  0    2
## 12  43   0  0      132  341   1       0     136     1     3.0     1  0    3
## 13  34   0  1      118  210   0       1     192     0     0.7     2  0    2
## 14  51   1  0      140  298   0       1     122     1     4.2     1  3    3
## 15  52   1  0      128  204   1       1     156     1     1.0     1  0    0
##    target
## 1       0
## 2       0
## 3       0
## 4       0
## 5       0
## 6       1
## 7       0
## 8       0
## 9       0
## 10      0
## 11      1
## 12      0
## 13      1
## 14      0
## 15      0
```

The head() function displays the first 15 rows of the heart dataset, allowing a quick preview of the data and any recent changes made to the columns.

5. Write a User-Defined Function Using Any Variable from the Dataset

```r
# Simple function to classify resting blood pressure
bp_category <- function(bp) {
  if (bp < 120) {
    return("Normal")
  } else if (bp < 140) {
    return("Elevated")
  } else {
    return("High")
  }
}

bp_category(130)
```

```
## [1] "Elevated"
```

A user-defined function called bp_category() was created to classify resting blood pressure into **Normal, Elevated, and High categories**. The function uses conditional statements to check the blood pressure value step by step: values **below 120** are classified as **Normal**, values from **120 to 139** are classified as **Elevated**, and values of **140 or above** are classified as **High**.

6. Use Data Manipulation Techniques and Filter Rows Based on Logical Criteria

```r
# filter high-risk male patients using sex, cholesterol, and age
high_risk_males <- heart %>%
  filter(sex == 1, chol > 240, age > 55)

# Printing results
cat("Number of high-risk male patients:", nrow(high_risk_males), "\n")
```

```
## Number of high-risk male patients: 164
```

```r
# viewing high_risk_males list
head(high_risk_males, 10)
```

```
##    age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal
## 1   58   1  0      114  318   0       2     140     0     4.4     0  3    1
## 2   56   1  2      130  256   1       0     142     1     0.6     1  1    1
## 3   70   1  2      160  269   0       1     112     1     2.9     1  1    3
## 4   59   1  0      138  271   0       0     182     0     0.0     2  0    2
## 5   64   1  0      128  263   0       1     105     1     0.2     1  1    3
## 6   67   1  0      100  299   0       0     125     1     0.9     1  2    2
## 7   59   1  3      170  288   0       0     159     0     0.2     1  0    3
## 8   59   1  0      170  326   0       0     140     1     3.4     0  0    3
## 9   56   1  0      125  249   1       0     144     1     1.2     1  1    2
## 10  65   1  0      110  248   0       0     158     0     0.6     2  2    1
##    target
## 1       0
## 2       0
## 3       0
```

```
## 4       1
## 5       1
## 6       0
## 7       0
## 8       0
## 9       0
## 10      0
```

This code uses filtering conditions to select only male patients with higher cholesterol and older age. These criteria were chosen to create a subgroup that may be at greater risk, which helps make the analysis more focused.

7. Identify the Dependent and Independent Variables, Use Reshaping Techniques, and Create a New Data Frame

```r
#Step 1: Identify the dependent variable
# In this dataset, target is the dependent variable because it shows
# whether heart disease is present or not.

dependentvariable <- as.data.frame(cbind(heart$target))
names(dependentvariable)[1] <- "Target"

# View the dependent variable
head(dependentvariable, n = 10)
```

```
##    Target
## 1       0
## 2       0
## 3       0
## 4       0
## 5       0
## 6       1
## 7       0
## 8       0
## 9       0
## 10      0
```

```r
# Step 2: Identify the independent variables
# These are the predictor variables we want to examine in relation to heart disease.
independentvariables <- heart %>%
  select(age, thalach, oldpeak, exang)

# View the independent variables
head(independentvariables, 10)
```

```
##    age thalach oldpeak exang
## 1   52     168     1.0     0
## 2   53     155     3.1     1
## 3   70     125     2.6     1
## 4   61     161     0.0     0
## 5   62     106     1.9     0
```

```
## 6   58     122     1.0     0
## 7   58     140     4.4     0
## 8   55     145     0.8     1
## 9   46     144     0.8     0
## 10  54     116     3.2     1
```

```r
# Step 3: Create a new data frame by combining the selected variables
# This new data frame includes both the dependent and independent variables.
new_dataframe <- heart %>%
  select(age, thalach, oldpeak, exang, target)

# View the new data frame
head(new_dataframe, 10)
```

```
##    age thalach oldpeak exang target
## 1   52     168     1.0     0      0
## 2   53     155     3.1     1      0
## 3   70     125     2.6     1      0
## 4   61     161     0.0     0      0
## 5   62     106     1.9     0      0
## 6   58     122     1.0     0      1
## 7   58     140     4.4     0      0
## 8   55     145     0.8     1      0
## 9   46     144     0.8     0      0
## 10  54     116     3.2     1      0
```

```r
# Step 4: Use reshaping techniques
# Convert the independent variables from wide format to long format
# so they are easier to compare in one column.

heart_long <- new_dataframe %>%
  pivot_longer(
    cols = c(thalach, oldpeak, exang),
    names_to = "variable",
    values_to = "value"
  )

# View the reshaped data
head(heart_long, 15)
```

```
## # A tibble: 15 x 4
##      age target variable value
##    <int>  <int> <chr>    <dbl>
##  1    52      0 thalach  168
##  2    52      0 oldpeak    1
##  3    52      0 exang      0
##  4    53      0 thalach  155
##  5    53      0 oldpeak    3.1
##  6    53      0 exang      1
##  7    70      0 thalach  125
##  8    70      0 oldpeak    2.6
##  9    70      0 exang      1
## 10    61      0 thalach  161
```

```
## 11    61      0 oldpeak    0
## 12    61      0 exang      0
## 13    62      0 thalach  106
## 14    62      0 oldpeak    1.9
## 15    62      0 exang      0
```

This code first identifies target as the dependent variable and then selects age, maximum heart rate, oldpeak, and exercise-induced angina as the independent variables. These variables were combined into a new data frame, and reshaping was used to change the data into long format so it would be easier to view and compare.

8. Remove Missing Values in the Dataset

```
colSums(is.na(heart))
```

```
##      age      sex       cp trestbps     chol      fbs  restecg  thalach
##        0        0        0        0        0        0        0        0
##    exang  oldpeak    slope       ca     thal   target
##        0        0        0        0        0        0
```

```
heart_clean <- na.omit(heart)

cat("Rows before:", nrow(heart), "| Rows after:", nrow(heart_clean))
```

```
## Rows before: 1025 | Rows after: 1025
```

This code checks the dataset for missing values and then removes any rows with incomplete data. A cleaned version of the dataset, called heart_clean, was created, and the number of rows before and after cleaning was displayed to show if any changes were made.

9. Identify and Remove Duplicated Data in the Dataset

```
cat("Number of duplicated rows:", sum(duplicated(heart_clean)), "\n")
```

```
## Number of duplicated rows: 723
```

```
heart_clean <- heart_clean[!duplicated(heart_clean), ]

cat("Rows after removing duplicates:", nrow(heart_clean))
```

```
## Rows after removing duplicates: 302
```

This code checks for duplicated rows in heart_clean, removes them, and then displays the number of rows left in the cleaned dataset.

10. Reorder Multiple Rows in Descending Order

```r
# sort by cholesterol and age
heart_sorted <- heart_clean %>%
  arrange(desc(chol), desc(age))

head(heart_sorted, 10)
```

```
##    age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal
## 1   67   0  2      115  564   0       0     160     0     1.6     1  0    3
## 2   65   0  2      140  417   1       0     157     0     0.8     2  1    2
## 3   56   0  0      134  409   0       0     150     1     1.9     1  2    3
## 4   63   0  0      150  407   0       0     154     0     4.0     1  3    3
## 5   62   0  0      140  394   0       0     157     0     1.2     1  0    2
## 6   65   0  2      160  360   0       0     151     0     0.8     2  0    2
## 7   57   0  0      120  354   0       1     163     1     0.6     2  0    2
## 8   55   1  0      132  353   0       1     132     1     1.2     1  1    3
## 9   55   0  1      132  342   0       1     166     0     1.2     2  0    2
## 10  43   0  0      132  341   1       0     136     1     3.0     1  0    3
##    target
## 1       1
## 2       1
## 3       0
## 4       0
## 5       1
## 6       1
## 7       1
## 8       0
## 9       1
## 10      0
```

This code creates a new data frame called heart_sorted by sorting heart_clean in descending order using arrange(desc(chol), desc(age)). This means the rows are ordered from highest to lowest cholesterol first, and then by highest to lowest age when cholesterol values are the same. The head(heart_sorted, 10) line displays the first 10 rows of the sorted dataset.

11. Rename Some of the Column Names in the Dataset

```r
heart_renamed <- heart_clean %>%
  rename(
    Age = age,
    Sex = sex,
    ChestPainType = cp,
    RestingBP = trestbps,
    Cholesterol = chol,
    FastingBS = fbs,
    RestECG = restecg,
    MaxHR = thalach,
    ExerciseAngina = exang,
    Oldpeak = oldpeak,
    ST_Slope = slope,
    NumVessels = ca,
```

```
    Thalassemia = thal,
    Target = target
  )

names(heart_renamed)
```

```
##  [1] "Age"            "Sex"            "ChestPainType"  "RestingBP"
##  [5] "Cholesterol"    "FastingBS"      "RestECG"        "MaxHR"
##  [9] "ExerciseAngina" "Oldpeak"        "ST_Slope"       "NumVessels"
## [13] "Thalassemia"    "Target"
```

This code creates a new data frame called heart_renamed by changing the original column names in heart_clean to clearer and more readable labels using rename(). For example, age is changed to Age, chol to Cholesterol, and target to Target. The names(heart_renamed) line then displays the updated column names to confirm that the renaming was applied correctly.

12. Add New Variables in the Data Frame Using a Mathematical Function

```
# create Age_Group
heart_clean$Age_Group <- case_when(
  heart_clean$age < 40 ~ "Young",
  heart_clean$age >= 40 & heart_clean$age < 60 ~ "Middle-aged",
  TRUE ~ "Senior"
)

# create cholesterol category and mathematical variables
heart_clean$chol_category <- case_when(
  heart_clean$chol < 200 ~ "Desirable",
  heart_clean$chol >= 200 & heart_clean$chol < 240 ~ "Borderline High",
  TRUE ~ "High"
)

heart_clean$chol_double <- heart_clean$chol * 2
heart_clean$bp_hr_ratio <- round(heart_clean$trestbps / heart_clean$thalach, 3)

# create a risk score
heart_clean$risk_score <- 0.3 * heart_clean$age +
                          0.4 * heart_clean$chol +
                          0.3 * heart_clean$trestbps

head(heart_clean[, c("age", "chol", "chol_double", "trestbps", "thalach", "bp_hr_ratio", "risk_score")])
```

```
##    age chol chol_double trestbps thalach bp_hr_ratio risk_score
## 1   52  212         424      125     168       0.744      137.9
## 2   53  203         406      140     155       0.903      139.1
## 3   70  174         348      145     125       1.160      134.1
## 4   61  203         406      148     161       0.919      143.9
## 5   62  294         588      138     106       1.302      177.6
## 6   58  248         496      100     122       0.820      146.6
## 7   58  318         636      114     140       0.814      178.8
## 8   55  289         578      160     145       1.103      180.1
```

```
## 9   46  249         498      120      144       0.833      149.4
## 10  54  286         572      122      116       1.052      167.2
```

This code adds several new variables to heart_clean. Age_Group and chol_category are created using case_when() to group age and cholesterol into categories. chol_double is created by multiplying cholesterol by 2, and bp_hr_ratio is calculated by dividing resting blood pressure by maximum heart rate and rounding the result to three decimal places. A risk_score variable is also created using a weighted combination of age, cholesterol, and resting blood pressure. The last line displays the first 10 rows of selected columns to show the newly added variables.

13. Create a Training Set Using Random Number Generator Engine

```
set.seed(1234)

train_index <- sample(1:nrow(heart_clean), size = 0.70 * nrow(heart_clean))
TrainingSet <- heart_clean[train_index, ]
TestingSet  <- heart_clean[-train_index, ]

dim(TrainingSet)
```

```
## [1] 211  19
```

```
dim(TestingSet)
```

```
## [1] 91 19
```

This code first uses set.seed(1234) to make the random sampling. It then uses sample() to randomly select 70% of the rows in heart_clean and stores their row numbers in train_index. Those rows are used to create TrainingSet, while the remaining rows are placed in TestingSet. The dim() function is then used to display the number of rows and columns in each dataset.

14. Print the Summary Statistics of the Dataset

```
summary(heart_clean)
```

```
##       age             sex               cp            trestbps
##  Min.   :29.00   Min.   :0.0000   Min.   :0.0000   Min.   : 94.0
##  1st Qu.:48.00   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:120.0
##  Median :55.50   Median :1.0000   Median :1.0000   Median :130.0
##  Mean   :54.42   Mean   :0.6821   Mean   :0.9636   Mean   :131.6
##  3rd Qu.:61.00   3rd Qu.:1.0000   3rd Qu.:2.0000   3rd Qu.:140.0
##  Max.   :77.00   Max.   :1.0000   Max.   :3.0000   Max.   :200.0
##       chol            fbs            restecg          thalach
##  Min.   :126.0   Min.   :0.000   Min.   :0.0000   Min.   : 71.0
##  1st Qu.:211.0   1st Qu.:0.000   1st Qu.:0.0000   1st Qu.:133.2
##  Median :240.5   Median :0.000   Median :1.0000   Median :152.5
##  Mean   :246.5   Mean   :0.149   Mean   :0.5265   Mean   :149.6
##  3rd Qu.:274.8   3rd Qu.:0.000   3rd Qu.:1.0000   3rd Qu.:166.0
```

```
##  Max.   :564.0   Max.   :1.000   Max.   :2.0000   Max.   :202.0
##      exang           oldpeak          slope             ca
##  Min.   :0.0000   Min.   :0.000   Min.   :0.000   Min.   :0.0000
##  1st Qu.:0.0000   1st Qu.:0.000   1st Qu.:1.000   1st Qu.:0.0000
##  Median :0.0000   Median :0.800   Median :1.000   Median :0.0000
##  Mean   :0.3278   Mean   :1.043   Mean   :1.397   Mean   :0.7185
##  3rd Qu.:1.0000   3rd Qu.:1.600   3rd Qu.:2.000   3rd Qu.:1.0000
##  Max.   :1.0000   Max.   :6.200   Max.   :2.000   Max.   :4.0000
##       thal           target       Age_Group         chol_category
##  Min.   :0.000   Min.   :0.000   Length:302         Length:302
##  1st Qu.:2.000   1st Qu.:0.000   Class :character   Class :character
##  Median :2.000   Median :1.000   Mode  :character   Mode  :character
##  Mean   :2.315   Mean   :0.543
##  3rd Qu.:3.000   3rd Qu.:1.000
##  Max.   :3.000   Max.   :1.000
##   chol_double      bp_hr_ratio       risk_score
##  Min.   : 252.0   Min.   :0.5250   Min.   :102.0
##  1st Qu.: 422.0   1st Qu.:0.7580   1st Qu.:139.0
##  Median : 481.0   Median :0.8655   Median :152.1
##  Mean   : 493.0   Mean   :0.9050   Mean   :154.4
##  3rd Qu.: 549.5   3rd Qu.:0.9928   3rd Qu.:167.4
##  Max.   :1128.0   Max.   :1.8220   Max.   :280.2
```

15. Perform Statistical Functions: Mean, Median, Mode, and Range

```
get_mode <- function(x) {
  uniq_vals <- unique(x)
  uniq_vals[which.max(tabulate(match(x, uniq_vals)))]
}

cat("=== Cholesterol ===\n")
```

```
## === Cholesterol ===
```

```
cat("Mean: ", mean(heart_clean$chol), "\n")
```

```
## Mean:  246.5
```

```
cat("Median: ", median(heart_clean$chol), "\n")
```

```
## Median:  240.5
```

```
cat("Mode: ", get_mode(heart_clean$chol), "\n")
```

```
## Mode:  204
```

```
cat("Range: ", range(heart_clean$chol), "\n\n")
```

```
## Range:  126 564
```

```r
cat("=== Maximum Heart Rate ===\n")
```

```
## === Maximum Heart Rate ===
```

```r
cat("Mean: ", mean(heart_clean$thalach), "\n")
```

```
## Mean:  149.5695
```

```r
cat("Median: ", median(heart_clean$thalach), "\n")
```

```
## Median:  152.5
```

```r
cat("Mode: ", get_mode(heart_clean$thalach), "\n")
```

```
## Mode:  162
```

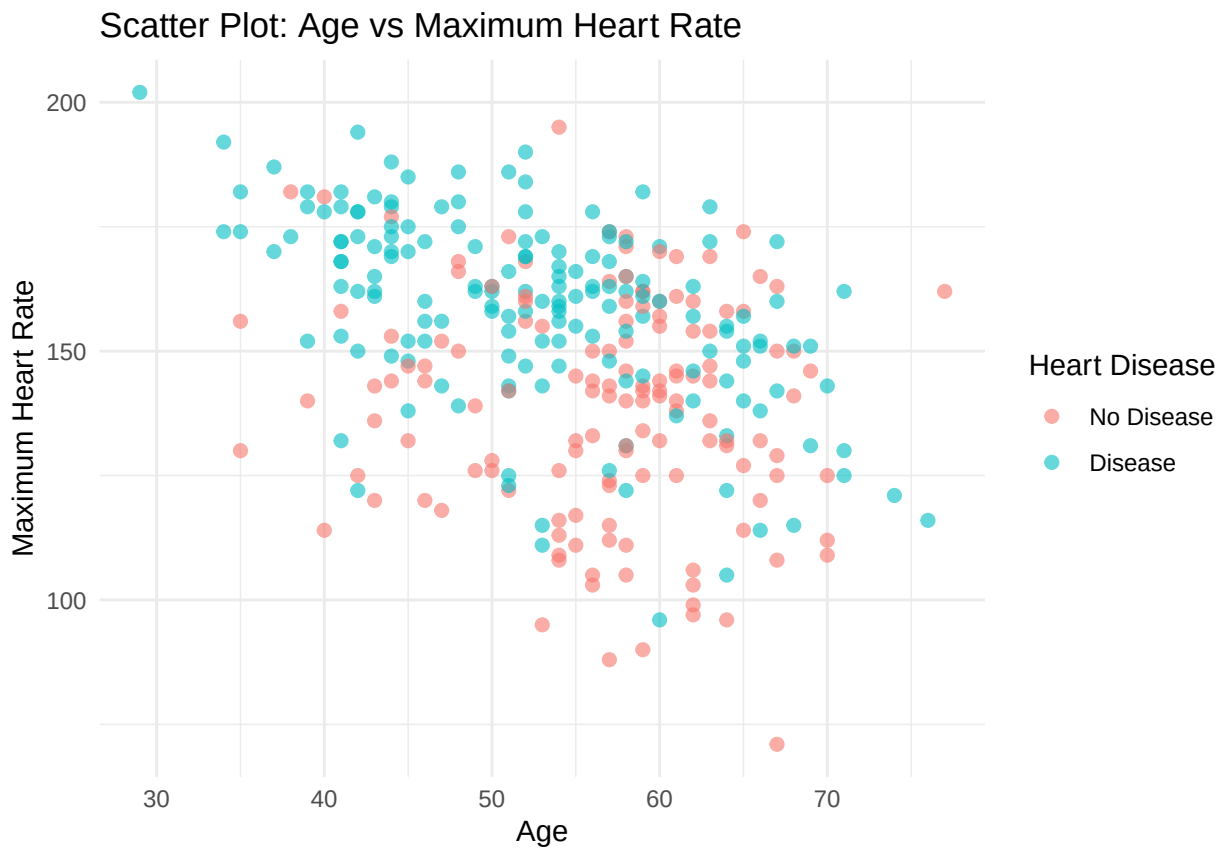
```r
cat("Range: ", range(heart_clean$thalach), "\n")
```

```
## Range:  71 202
```

This code first defines a custom function called get_mode() to find the mode of a numeric variable. It then uses mean(), median(), get_mode(), and range() to calculate the mean, median, mode, and range for cholesterol and maximum heart rate in heart_clean. The cat() function is used to print the results in a clear labeled format.

16. Plot a Scatter Plot for Any 2 Variables in the Dataset

```r
ggplot(heart_clean, aes(x = age, y = thalach, color = as.factor(target))) +
  geom_point(alpha = 0.6, size = 2) +
  labs(
    title = "Scatter Plot: Age vs Maximum Heart Rate",
    x = "Age",
    y = "Maximum Heart Rate",
    color = "Heart Disease"
  ) +
  scale_color_discrete(labels = c("No Disease", "Disease")) +
  theme_minimal()
```



Analysis: Age vs. Maximum Heart Rate

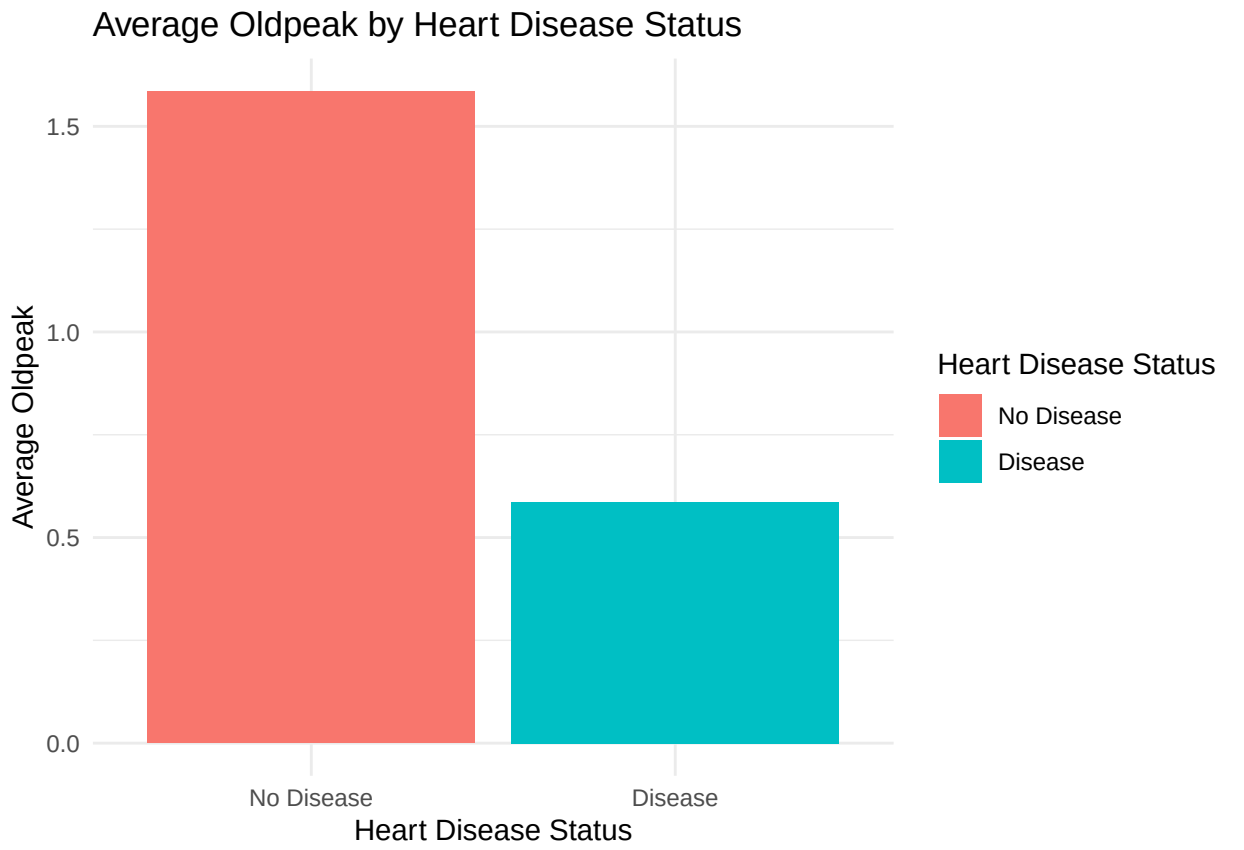
The group examined the relationship between age and maximum heart rate because both are continuous variables and are known to be related in cardiovascular function. The scatter plot shows a downward trend, indicating an inverse relationship between age and maximum heart rate. As age increases, maximum heart rate tends to decrease. The points are somewhat spread out, suggesting that the relationship is moderate rather than strong. This means that while there is a general pattern, there is still variability among individuals. The color grouping shows that both patients with and without heart disease generally follow a similar downward trend, although the distribution of points suggests some differences between the groups.

17. Plot a Bar Plot for Any 2 Variables in the Dataset

```
bar_data <- heart_clean %>%
  group_by(target) %>%
  summarise(avg_oldpeak = mean(oldpeak))

ggplot(bar_data, aes(x = factor(target), y = avg_oldpeak, fill = factor(target))) +
  geom_bar(stat = "identity") +
  labs(
    title = "Average Oldpeak by Heart Disease Status",
    x = "Heart Disease Status",
    y = "Average Oldpeak",
    fill = "Heart Disease Status"
  ) +
```

```
  scale_x_discrete(labels = c("No Disease", "Disease")) +
  scale_fill_discrete(labels = c("No Disease", "Disease")) +
  theme_minimal()
```



Analysis: Average Oldpeak by Heart Disease Status

Oldpeak represents ST depression during exercise relative to rest, which reflects how the heart responds under stress. The bar plot compares the average oldpeak between patients with and without heart disease. In this dataset, patients without heart disease appear to have a higher average oldpeak compared to those with heart disease. This pattern is somewhat unexpected, as higher oldpeak is typically associated with cardiac stress (Kashou et al., 2023). This suggests that oldpeak alone may not be a strong predictor of heart disease and highlights the importance of considering multiple variables when analyzing cardiovascular risk.

Source: Kashou, A. H., Basit, H., & Malik, A. (2023). ST segment. In StatPearls. StatPearls Publishing. https://www.ncbi.nlm.nih.gov/books/NBK459364/

18. Find the Correlation Between Any 2 Variables by Applying Pearson Correlation

```
# age vs maximum heart rate
cor_value <- cor(heart_clean$age, heart_clean$thalach, method = "pearson")
cor_value
```

```
## [1] -0.3952352
```

Analysis:

To quantify the relationship observed in the scatter plot, the group applied Pearson correlation between age and maximum heart rate. The correlation coefficient is **-0.3952352**, indicating a moderate negative relationship. This means that as age increases, maximum heart rate tends to decrease. In simpler terms, older patients in this dataset generally had lower maximum heart rates than younger patients. Although the relationship is not strong, it is consistent enough to show a noticeable pattern.

Conclusion

Based on the group analysis, heart disease in this dataset does not seem to be explained by just one factor like cholesterol. Instead, it becomes clearer when looking at multiple variables together, especially age, resting blood pressure, maximum heart rate, exercise-induced angina, and oldpeak. These results suggest that cardiovascular and exercise-related variables can help show differences between patients with and without heart disease, while also showing how some variables are related to each other. For example, the moderate negative correlation between age and maximum heart rate suggests that older patients generally had lower maximum heart rates in this dataset. Overall, the findings show that heart disease is influenced by a combination of factors rather than a single variable.